



Región Crítica & Región Crítica Condicional

Región Crítica (RC)



Una **región crítica** es un mecanismo de control de la concurrencia donde hacemos que el compilador se encargue del control de la exclusión mutua a través de la definición de dicha región. Mientras un proceso está ejecutando la región, ningún otro puede entrar en ella.

Explicación

Consiste en **encapsular el acceso a un recurso compartido** dentro de una construcción especial (conceptualmente indicada como `region`).

Solo por el hecho de definir esta región:

-  Se garantiza **exclusión mutua**.
-  No se garantiza **sincronización**.
-  No se evita el **interbloqueo**.
-  No se asegura **vivacidad**.

Aseguramos exclusión mutua, no vivacidad.

El compilador o el sistema de ejecución se encargan internamente de usar los mecanismos necesarios (cerrojos, instrucciones atómicas, etc.), sin que el programador tenga que gestionarlos explícitamente.

`region recurso:`

`# código que accede al recurso compartido`

```
recurso := recurso + 1
end region
```

Esta es una primitiva teórica. A nivel de implementación no tenemos instrucciones o clases específicas que sean “regiones críticas”, pero sí que podemos simular su comportamiento.

Java

```
public class Contador {
    private int valor = 0; // recurso compartido

    // region contador do valor := valor + 1
    public synchronized void incrementar() {
        valor++;
    }
    // end region
}
```

C++

```
#include <mutex>

class Contador {
public:
    void incrementar() {
        std::lock_guard<std::mutex> guard(m);
        ++valor;
    }

private:
    int valor = 0; // recurso compartido
    std::mutex m; // cerrojo de la región
};
```

Como vemos, simplemente hemos agrupado el acceso a recursos compartidos en regiones.

Si queremos intentar garantizar vivacidad debemos utilizar condiciones de sincronización, y para ello nacen las Regiones Críticas Condicionales (RCC).

Aunque estas tampoco son un mecanismo perfecto de vivacidad, ya que podríamos diseñar malas condiciones de sincronización que nos llevaran a inanición o interbloqueos.

Región Crítica Condicional (RCC)



Una **Región Crítica Condicional (RCC)** es una región crítica que **solo puede ejecutarse cuando una condición lógica es verdadera**. Si no se cumple, el proceso se bloquea hasta que lo sea

Explicación

A nivel teórico, Hoare la define como *"una sección crítica que SOLO se ejecuta cuando una condición asociada es verdadera; si no lo es, el proceso se bloquea hasta que dicha condición pase a ser cierta"*.

```
region recurso when condición do  
  CÓDIGO  
end
```

Para poder implementarla necesitaríamos una forma de declarar la región, y una variable de condición.

La sección crítica **solo se ejecuta si** `condición = true` .

Modelo teórico

Toda RCC tendrá siempre 2 colas asociadas:

- **La de exclusión mutua**, a la que entrarán todas las entidades concurrentes que quieran acceder a la sección.

- **La de eventos** (o la cola asociada a la variable de condición), donde bloquearemos los procesos que no cumplan la condición de la variable de condición.

Por definición teórica pura, solo tendríamos una variable de condición.

```
region recurso when buffer_no_lleno do
  producir(x)
end region
```

Si quieres más condiciones, había que combinarlas con AND, OR, etc.

```
region buffer when (hay_hueco AND consumidor_listo) do
  producir(x)
end region
```

Si queremos más variables de condición habría que usar monitores.

```
monitor Buffer:

  condition no_lleno
  condition no_vacio
  buffer, count

  procedure put(x):
    if count == N:
      wait(no_lleno)
    buffer.add(x)
    count++
    signal(no_vacio)

  procedure get():
    if count == 0:
      wait(no_vacio)
    y = buffer.remove()
    count--
```

```
signal(no_lleno)
return y
```

¿Cómo funcionan estas colas?

Estas 2 colas que tiene toda región crítica generan una prioridad asociada, ya que el flujo de ejecución sería:

Para entender su funcionamiento, debemos ver qué pasa al intentar entrar a esta sección crítica, y al salir de ella:

Entrada a la región crítica

Cuando una entidad concurrente quiere ejecutar la RCC:

1. Intenta obtener la **exclusión mutua** sobre la región.
 - Si **no lo consigue** → se bloquea en la **cola de exclusión mutua**.
 - Si **lo consigue** → entra en la región y pasa al siguiente paso.
2. Ya dentro de la región, se **evalúa la condición** asociada a la RCC.
 - Si la condición es **true** → ejecuta la **sección crítica**.
 - Si la condición es **false**:
 - se bloquea en la **cola de eventos**.
 - **libera la exclusión mutua** (para que otros puedan entrar).
 - y esperará a que el sistema lo despierte cuando la condición pueda volver a ser verdadera.

Salida de la región crítica

Cuando un proceso termina de ejecutar la sección crítica y va a salir de la región:

1. El sistema mira primero la **cola de eventos**:
 - Revisa si alguno de los procesos bloqueados allí **ya cumple la condición** (se reevalúan sus condiciones).
 - Si encuentra alguno que la cumpla:
 - lo despierta y le **cede la exclusión mutua** a ese proceso.

- Ese proceso pasa directamente a ejecutar su sección crítica.
2. Solo si **ningún proceso** en la cola de eventos puede entrar (ninguno cumple la condición):
- el sistema desbloquea al **siguiente proceso** de la **cola de exclusión mutua**,
 - y este será el siguiente en entrar a la región.

Este comportamiento de "salida" se produce siempre que se libera la exclusión mutua. Es decir, se aplica tanto cuando una entidad:

- Termina la ejecución del código de la sección crítica, como cuando
- Una entidad ha obtenido la exclusión mutua pero, al evaluar la condición, esta resulta falsa y se bloquea en la cola de eventos, liberando la exclusión mutua.



Debido a este orden, la **cola de eventos TIENE PRIORIDAD sobre la cola de exclusión mutua**.

Este es el motivo por el que no podemos realizar una implementación teórica de lo que propuso Hoare en lenguajes como Java o C++, ya que en dichos lenguajes no podemos garantizar ni hablar de prioridades por su carácter generalista.

Como muchos podemos realizar una **SIMULACIÓN DE IMPLEMENTACIÓN**, pero jamás implementar la teórica.

Podemos hacerlo usando semáforos o monitores:

Monitores

monitor Recurso:

```
var recurso
var cond : condition

procedure usar_recurso():
  if not CONDICIÓN then
```

```

    wait(cond)      // pasa a la cola de eventos
end if

// SECCIÓN CRÍTICA
USAR recurso

signal(cond)       // puede cambiar la condición

```

Semáforos

```

semaphore mutex = 1
semaphore condSem = 0
int waitCount = 0

procedure enter_RCC():
    wait(mutex)

    while not CONDICIÓN do
        waitCount := waitCount + 1
        signal(mutex)
        wait(condSem)
        wait(mutex)
        waitCount := waitCount - 1
    end while

procedure exit_RCC():
    if waitCount > 0 then
        signal(condSem) // prioridad a la cola de eventos
    else
        signal(mutex) // cola de exclusión mutua
    end if

enter_RCC()
    SECCIÓN_CRÍTICA
exit_RCC()

```

Java (API Standart)

En la API estándar de Java:

-  **No existen variables de condición como objetos independientes.**
-  Cada objeto tiene:
 - un **cerrojo implícito** (usado con `synchronized`),
 - un **único wait-set** (cola de espera).

La sincronización de la condición se implementa mediante los métodos de `Object`:

- `wait()` → bloquea el hilo y lo mueve al **wait-set** del objeto.
- `notify()` → despierta a **un hilo arbitrario** del wait-set.
- `notifyAll()` → despierta a **todos los hilos** del wait-set.

⚠ Si el wait-set está vacío:

| `notify()` y `notifyAll()` no tienen efecto → la señal se pierde.

```
public class Buffer {  
  
    private int dato;  
    private boolean lleno = false;  
  
    public synchronized void producir(int x) throws InterruptedException {  
        while (lleno) {  
            wait(); // condición falsa → cola de eventos (wait-set)  
        }  
  
        // Sección crítica  
        dato = x;  
        lleno = true;  
  
        notifyAll(); // cambia la condición de los consumidores  
    }  
  
    public synchronized int consumir() throws InterruptedException {
```



```

while (!llen) {
    wait(); // condición falsa → cola de eventos
}

int x = dato;
llen = false;

notifyAll(); // cambia la condición de los productores
return x;
}
}

```

Java (API Alto Nivel)

Con `ReentrantLock` y `Condition`:

-  Una **variable de condición es un objeto real** (`Condition`).
-  Cada `Condition` tiene su **propia cola de espera**.
-  Se pueden definir **múltiples condiciones por monitor**.

Operaciones principales:

- `await()` → equivalente a `wait()`
- `signal()` → equivalente a `notify()`
- `signalAll()` → equivalente a `notifyAll()`

```

import java.util.concurrent.locks.*;

public class Buffer {

    private int dato;
    private boolean llen = false;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition noLlen = lock.newCondition();
    private final Condition noVacio = lock.newCondition();
}

```

```

public void producir(int x) throws InterruptedException {
    lock.lock();
    try {
        while (llenado) {
            noLlenado.await();
        }

        dato = x;
        llenado = true;

        noVacio.signal(); // despierta consumidores
    } finally {
        lock.unlock();
    }
}

public int consumir() throws InterruptedException {
    lock.lock();
    try {
        while (!llenado) {
            noVacio.await();
        }

        int x = dato;
        llenado = false;

        noLlenado.signal(); // despierta productores
        return x;
    } finally {
        lock.unlock();
    }
}
}

```

C++

En C++ ocurre lo mismo que en Java alto nivel:

- Se usan:
 - `std::mutex`
 - `std::condition_variable`
-  Cada `condition_variable` es una **variable de condición real**.
-  Permite múltiples condiciones por monitor.
-  No garantiza la semántica Hoare pura (prioridad de la cola de eventos).

```
#include <mutex>
#include <condition_variable>

class Buffer {
private:
    int dato;
    bool lleno = false;

    std::mutex m;
    std::condition_variable noLleno;
    std::condition_variable noVacio;

public:
    void producir(int x) {
        std::unique_lock<std::mutex> lock(m);

        while (lleno) {
            noLleno.wait(lock);
        }

        dato = x;
        lleno = true;

        noVacio.notify_one();
    }
};
```

```

}

int consumir() {
    std::unique_lock<std::mutex> lock(m);

    while (!lleno) {
        noVacio.wait(lock);
    }

    int x = dato;
    lleno = false;

    noLleno.notify_one();
    return x;
}
};

```

If VS While

En la teoría podemos modelar la evaluación de la condición usando un simple `if`, en la práctica SIEMPRE habrá que usar un `while`.

En teoría basta:

```
if (condición) ejecutar
```

En implementación **siempre debe usarse** `while`, porque:

- Puede haber:
 - despertares espurios,
 - señales incorrectas,
 - competencia por el cerrojo.

Por eso:

✅ En código real, **SIEMPRE** while.